

SORTNET

FASE II

Simon Holm, Johannes Hesselager Rothe & Anne Sofie Nielsen Høj

sihol24, jorot24 & anho024

Fasekoordinator Simon Holm

DM574 - gruppe 15

29. November 2024

1 Introduktion

Denne rapport omhandler anden fase sorteringsnetværk projektet. Projektet er et afsluttende gruppeprojekt i faget introduktion til programmering. I projektet anvendes Python som programmeringssprog. I projektet er målet at udvikle de moduler, der sammen skaber et sorteringsnetværk baseret på et specifikt antal kanaler. Projektet lægger vægt på strukturering samt opbygning, hvor det fordeles mellem tre moduler: `generate.py`, `prune.py` og `network_finder.py`. Modul `prune.py` blev udleveret som en færdig komponent og skal dermed integreres med de øvrige moduler. For at understøtte udviklingen skal der også implementeres en ny datatype, som defineres og tilbydes i modulet `filter.py`. Denne datatype vil spille en afgørende rolle i filtrering i sorteringsnetværket.

Formålet med dette projekt er at kunne skabe filtre der repræsenterer ikke færdige sorteringsnetværk. Her bruges "extend" til at udvide mængden af filtre ved nye filtre der bliver lavet, baseret på de gamle. Dog med tilføjede ikke overflødige comparators på alle led. Dette sættes derefter sammen med programmet `network_finder`, der gentager denne proces indtil, en af de udvidede filtre er et sorteringsnetværk

2 Programopbygning

Denne fase er opdelt i tre moduler, der er henholdsvis `filter.py`, `generate.py` og `network_finder.py`, som samlet set har 8 funktioner.

`Filter.py` indeholder skabelsen og ændringen af dataclassen `filter`, såsom at tilføje comparators til filtre og tjekke om de er overflødig i forhold til filtrene. Dermed kan det vurderes hvilke der er overflødige, og derigennem opnås en mere effektiv proces, uden adskillige gentagelser da `filterclassen` indeholder outputtet af filteret. Dette modul indeholder blandt andet "is_redundant" samt "add" funktioner.

`Generate.py` indeholder kun en funktion "extend", som ved input af en liste af filtre, tilføjer alle ikke overflødige comparators på alle måder til alle filtre, og dermed genererer en ny liste med alle de nye filtre. Hvorefter de alle bliver testet i forhold til deres brugbarhed med de enkle comparators. Det sidste modul `network_finder.py` indeholder det ene program, der ved hjælp af et input "n", skaber et sorteringsnetværk på "n" kanaler baseret på extend metoden fra `generate`. Dette gentages indtil listen indeholder et sorteringsnetværk, hvorefter pythons implementering af dette udskrives. Her bruges `to_program` fra fase 1.

2.1 Programmeringsparadimer

Dette projekt har primært brugt imperativ og funktionel programmering. Dette er grundet i, at gruppens erfaring og evner er bedre i disse end i rekursiv programmering. Gruppen vurderede desuden, at der ikke var nogen funktioner, hvor en rekursiv tilgang ville give bedre resultater, der kunne opveje forskellene i evner mellem de forskellige paradigmer.

Herunder bruges while-løkkerne igen, med succes. Da det vurderes af gruppen, til at give god læselighed samt er en effektiv måde at implementere programmerne. Dette er især relevant i situationer hvor der skal bruges nestede løkker. I disse situationer har gruppen vurderet at det giver programmet en bedre struktur og læselighed hvis der bruges nestede while-løkker frem for funktionelle løsninger.

Da de endelige funktioner tager meget lang tid, har det være vigtigt at undgå unødvendige handlinger. Dette inkluderer scenarier hvor, en funktion skal bruge en hjælpefunktion, men outputtet ikke ændres efterfølgende. Her ville den sekundære funktion blive kaldt i starten, hvor dens værdi bliver gemt i variabel, som den så kan tjekkes på, i stedet for at kalde funktionen flere gange.

Gruppen har forsøgt at undgå Pythons indbyggede funktioner. I stedet for har der været lagt fokus på at bruge funktionerne der er blevet lært og øvet til forelæsninger og øvelsestimuler. Det vil lede til at nogle af funktionerne kunne løses mere effektivt og resolut, hvis der blev brugt Pythons funktioner, men da formålet med dette projekt er at forstå de givne værktøjer, er dette hvor gruppen har lagt vægt.

2.2 Hjælpefunktioner

Gruppen har diskuteret brugen af hjælpe funktioner ved projektets begyndelse. Der blev gruppen enige om det blev vurderet, hvis en funktion var meget lang, eller det ville forbedre funktions læselighed med en hjælpefunktion, ville det blive taget op i gruppen. I disse 3 moduler, blev der ikke behov for mange hjælpefunktioner i forhold til indførelse af programmet.

3 Programmer

Programmets funktioner er, som tidligere nævnt, opdelt i 3 Python filer, filter.py, generate.py og network_finder.py.

3.1 Filter.py

3.1.1 Dataclass - Filter

Datotypen Filter er i dette projekt lavet som en dataclass. Det gøres fordi projektet bruger 2 forskellige attributter. Desuden synes gruppen, at det giver bedre læselighed.

3.1.2 Make_empty_filter

Make_empty_filter kalder først empty_network fra fase 1 netværks modul. Således kan funktionen derefter returnerer et filter, bestående af et nyt tomt filter samt alle outputs der passer til antallet af kanaler.

3.1.3 Net

Net funktionen returnerer netværket i et filter.

3.1.4 Out

Out funktionen returnerer, ligesom net funktionen, outputtene i et filter.

3.1.5 Is_redundant

Is_redundant funktionen skal teste om en given comparator er overflødig, dermed om den ændrer noget på den endelige sorterings evne. Ved at bruge funktion apply_comparator på alle de binære lister, med den comparator der skal testes. Vil der blive genereret en ny liste med de opdaterede outputs hvis comparatoren blev tilføjet til filteret. Så kan de outputs fra det uændrede filter sammenlignes med de outputs påvirket af comparatoren, og hvis de er ens, så er comparatoren redundant.

```
o = []
i = 0
lenout = len(f.output)
while i < lenout:
    o = o + [(apply_comparator(c,f.output[i]))]
    i = i + 1
return o == f.output
```

Bemærk at længden af output listen bliver gemt som en variabel der kan bruges gentagende gange uden at skulle bruge funktionen.

Is_redundant testes for at undgå uheldige og uventede inputs i funktionen. Det gøres ved, for eksempel at teste comparatorer som med sikkerhed er overflødige. Eksempelvis med en comparator (0,1) på en liste med 2 kanaler og ingen comparatorer: [[0, 0], [0, 1], [1, 0], [1, 1]] hvorefter funktionen returnerer false.

Det kan være en fordel at sikre yderligere om funktionen kan fungere med uventede inputs såsom negative tal. Dog har tidligere funktioner, f.eks. comparator i fase 1, elimineret disse muligheder i kontrakten. Det ville også give en fejl hvis der blev testet med en comparator der har kanalerne 1 og 47 på et filter med 2 kanaler, derfor er det specificeret i kontrakten at kanalerne på comparatoren skal være inden for kanalerne på filtret.

3.1.6 Add

For at tilføje en comparator til et filter, skal der både tilføjes en comparator til netværket, samt outputtet i filteret skal opdateres. Den første ide til at lave den funktion, indebærer at tilføje comparatoren til netværket i filterne, og så derefter sætte netværket ovenpå outputtet efterfølgende. Efter en kort diskussion blev denne ide skrottet da det ville være meget ineffektivt at sætte hele netværket ovenpå outputtet, når det kun er en enkel comparator der tilføjes. Det ledte til en opdateret løsning hvor den nye comparator bliver tilføjet til netværket, og derefter sat ovenpå alle outputs i filtret.

For at tilføje comparatoren til netværket, laves der først et nyt netværk, hvor netværket og dets output bliver sat lige med det netværk, hvor der skal tilføjes en comparator. Derefter kan append funktionen bruges, til at tilføje en comparator til netværket i filteret. Dette gøres for at undgå bivirkninger, da funktionen skal returnere en kopi af det gamle filter, men med en tilføjet comparator.

Outputs i filteret bliver opdateret ved at anvende en while løkke der kører igennem for længden af sub-liste i outputs. For hver sub-liste anvendes `apply_comparator` på den pågældende liste med den comparator der skal tilføjes til filteret. Derved bliver den associerede outputliste opdateret. I `add` kunne der også opstå et problem, hvis der blev tilføjet en comparator med kanalerne 1 og 47 på et filter med 2 kanaler, derfor specificeres det i kontrakten på funktionen, at kanalerne på comparatoren skal være indenfor kanalerne på filtret.

For at undgå bivirkninger i denne funktion, er der blandt andet blevet testet, ved at skabe et filter, og derefter sætte et nyt filter lig med det gamle filtre, hvor der så er blevet tilføjet en comparator:

```
f1 = make_empty_filter(2)
f2 = add(comparator(0,1),f1)
print(f1)
print(f2)
```

Her vil output være:

Filter(net=[], output=[[0, 0], [0, 1], [1, 0], [1, 1]])

Filter(net=[comparator(i=0, j=1)], output=[[0, 0], [0, 1], [0, 1], [1, 1]])

Og dermed kan med denne metode undgå at skabe bivirkninger.

3.1.7 Is_sorting

Programmet fra fase 1 har allerede en eksisterende funktion, der tjekker om et netværk er et sorterings-netværk. Den funktion kan nu anvendes igen når et netværk i et filter skal tjekkes på samme måde. Her kan funktionen anvendes på netværket i filteret. Her sættes netværket samt længden af outputs ind.

3.2 Generate.py

3.2.1 Extend

Extend-funktionen har til opgave at udvide et filter med alle mulige ikke overflødige comparatorer. Funktionen starter med et tomt filter, hvilket er udgangspunktet i funktionen.

Funktionen bruger 2 while-løkker. For hvert filter i listen med filtre, skal funktionen gøre følgende: For hver comparator i en liste med "n" standard comparators, skal funktionen tjekke om comparatoren er overflødig i forhold til filterets outputs. Hvis comparatoren ikke er overflødig, tilføjes den til den nye liste.

Dette leder til en overordnet struktur der tester alle de standard comparators på liste med n kanaler, på alle de filtre i en liste og returnerer en ny list med kun nye filtre der alle har fået tilføjet en comparator i forhold til den tidligere generation. Gruppen har bevidst valgt at holde funktionen så simpel som muligt, da den skal køres mange gange i network_finder. Samt taget en beslutning af extend, så den ikke returnerer de gamle filtre der ikke er blevet udvidet. Dette er gjort da extend bruges til at genererer et sorteringsnetværk, og hvis den tidligere generation ikke indeholder et sorteringsnetværk, hvorefter der kaldes et nyt extend, hvor det ikke giver mening at beholde overflødige filtre der skal tjekkes ekstra i den del af network_finder.

```
comp_length = len(list_comparator)
    filt_length = len(w)
    while i < filt_length:
        while j < comp_length:
```

For hvert kald af extend, bliver antallet af comparator og filtre gemt i en separat variabel så den ikke skal beregne det flere gange.

Funktionen kan testes med at extende på et enkelt tom filter med 3 kanaler. De standard comparators på et netværk med 3 kanaler er [0,1], [0,2] og [1,2]. Derved ville funktionen skulle returnere end liste med 3 filtre, hver med en af kanalerne (da ingen af dem er redundante):

```
f = [make_empty_filter(3)]
print(extend(f,3))
```

Som giver outputtet (hvilket er korrekt):

```
[Filter(net=[comparator(i=0, j=2)], output=[[0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1, 1],
[0, 0, 1], [1, 0, 1], [0, 1, 1], [1, 1, 1]]),
Filter(net=[comparator(i=0, j=1)], output=[[0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1, 1],
[0, 1, 0], [0, 1, 1], [1, 1, 0], [1, 1, 1]]),
Filter(net=[comparator(i=1, j=2)], output=[[0, 0, 0], [0, 0, 1], [0, 0, 1], [0, 1, 1],
[1, 0, 0], [1, 0, 1], [1, 0, 1], [1, 1, 1]])]
```

3.3 Network_finder.py

Network_finder er et program der skal finde et sorteringsnetværk på n-kanaler. Det gør den ved at anvende "extend" funktionen flere gange på en liste med filters. Således genereres en voksende liste af filter hvor hver ny generation indeholder en ekstra ikke overflødig comparator. Når der er blevet genereret en ny generation af filtre, bliver listen testet for at tjekke, om en af dem er et sorteringsnetværk med funktionen is_sorting fra filter. Efterfølgende bliver listen behandlet af funktionen prune, som i denne del af projektet returner listen uændret.

Der skulle også laves en måde hvorpå, hvis programmet finder et sorteringsnetværk, at den ikke skulle tjekke resten af filtrene. "Break" virkede som en utilfredsstillende måde, så efter en diskussion af forskellige løsninger, hvorefter en simpel men effektiv løsning blev valgt. Funktionen tjekker alle filtre, så længe kontrolvariablen, der også styrer hvilket filter der tjekkes, er under mængden af filtre. Så hvis funktionen finder et sorteringsnetværk, så tilføjes mængde af filtre til kontrolvariablen, hvorefter while løkken stopper.

Der blev også udtænkt en anden mulig løsning, til at vurdere om et filter er et sorteringsnetværk. Is_sorting tjekker om alle de binære lister er sorteret bit efter bit. Men ved at lave allerede sorteret liste på samme længde som filtrene, ville der kun skulle sammenlignes med den liste. Den ide blev dog skrottet, da hvis bare en af de binære lister skiftede plads, altså hvis filtret var stadig et sorteringsnetværk, ville løsningen ikke fungere. Så den ville fungere hvis der blev brugt vores løsninger i fase 1, men ikke hvis der blev brugt andre.

Hvis et filter har et sorteringsnetværk, bliver filteret tilføjet til en kontrol liste, der bruges til at styre hvornår programmet skal gå videre. Så længe listen er tom, fortsætter extend, hvor den tjekker om det er sorteringsnetværk. Således fortætter funktionen med, at extend indtil et sorteringsnetværk bliver fundet. Når listen indeholder et filter, bliver der printet information om processen, samt to_program med sorteringsnetværket:

```
What should the size of the network be?
4
The size of the network is: 4
Number of extends: 5
Size: 5
Found a sorting network on 4 channels with size 5
The Python implementation for sorting a list v of size 4 is:
['if v[0] > v[3]:', 'aux = v[0]', 'v[0] = v[3]', 'v[3] = aux', 'if v[1] > v[2]:',
'aux = v[1]', 'v[1] = v[2]', 'v[2] = aux', 'if v[0] > v[1]:', 'aux = v[0]', 'v[0] = v[1]',
'v[1] = aux', 'if v[2] > v[3]:', 'aux = v[2]', 'v[2] = v[3]', 'v[3] = aux', 'if v[1] > v[2]:',
'aux = v[1]', 'v[1] = v[2]', 'v[2] = aux']
```

Her ses et eksempel hvor network_finder hvor der vælges 4 kanaler. Da network_finder starter med et tomt filter bliver der lavet 5 extensions, så det endelige sorteringsnetværk indeholder 5 comparators.

4 Konklusion

I vores tidligere fase, fase 1, lavede vi et sorteringsnetværk. I denne fase 2 har gruppen udarbejdet i alt 8 funktioner der sammen finder et gyldigt sorteringsnetværk på et given antal kanaler. I projektets fase 2 bliver den anvendte prune-funktion brugt som placeholder til fase 3. Det betyder for `generate_and_prune` funktionen at køretiden bliver meget eksponentiel. Derfor er det for fase 2 urealistisk at køre programmet med mere end 4-5 kanaler. Gruppen har formået at køre programmet på 2, 3 samt 4 kanaler på meget kort tid, få sekunder. Dog på 5 kanaler tog det et stykke tid, for at få et forsøg på et gyldigt sorteringsnetværk.

5 Bilag

5.1 Filter.py

```
from dataclasses import dataclass
from network import (Network, empty_network,
append, is_sorting as is_sorting_network, all_outputs)
from comparator import (comparator, apply as apply_comparator)
@dataclass
class Filter:
    net: Network
    output: list[list[int]]

def make_empty_filter(n:int) -> Filter:
    """Returns a filter containing all binary inputs of a given length.
    >>>make_empty_filter(2)
    Filter(net=[], output=[[0, 0], [0, 1], [1, 0], [1, 1]])

    """
    n1 = empty_network()
    return Filter(n1,all_outputs(n1,n))

def net(f:Filter) -> Network:
    """Returns the network in a filter.
    >>>net(Filter(net=[comparator(0,1)], output=[[0, 0], [0, 1], [1, 0], [1, 1]]))
    Filter(net=[comparator(0,1)])

    """
    f1 = f.net
    return f1

def out(f:Filter) -> list[list[int]]:
    """Returns the outputs in a filter.
    >>>out(Filter(net=[], output=[[0, 0], [0, 1], [1, 0], [1, 1]]))
    [[0, 0], [0, 1], [1, 0], [1, 1]]

    """
    f1 = f.output
    return(f1)

def is_redundant(c:comparator, f:Filter) -> bool:
    """Checks if redundant in relation to a filter. The channels on the comparator has within
    channel size of the filter.
    >>>is_redundant(comparator(0,1), Filter(net=[], output=[[0, 0], [0, 1], [1, 0], [1, 1]]))
    False

    """
    o = []
    i = 0
    lenout = len(f.output)
    while i < lenout:
        o = o + [(apply_comparator(c,f.output[i]))]
        i = i + 1
    return o == f.output
```



```

def add(c:comparator, f:Filter) -> Filter:
    """Adds a comparator on a network in the filter. The channels on the comparator has within
    channel size of the filter.
    >>>add(comparator(0,1), Filter(net=[], output=[[0, 0], [0, 1], [1, 0], [1, 1]]))
    Filter(net=[comparator(0,1)], output=[[0, 0], [0, 1], [1, 0], [1, 1]])
    """

    new_filter = make_empty_filter(1)
    new_filter.output = f.output[:]
    new_filter.net = f.net[:]
    new_filter.net = append(c,new_filter.net)
    for i in range(len(new_filter.output)):
        new_filter.output[i] = (apply_comparator(c,new_filter.output[i]))
    return new_filter

def is_sorting(f:Filter) -> bool:
    """Checks if the network in the filter is a sortingnetwork. """
    return is_sorting_network(f.net,len(f.output[0]))

def sorted_list(n:int) -> list[list[int]]:
    """Generates a sorted list of size n"""
    w = all_outputs([],n)
    i = 0
    aux = 0
    while i < len(w):
        j = 0
        while j < n-1:
            if w[i][j] > w[i][j+1]:
                aux = w[i][j]
                w[i][j] = w[i][j+1]
                w[i][j+1] = aux
                j = 0
            else:
                j = j + 1
        i = i + 1
    return(w)

```

5.2 Generate.py

```
from filter import Filter
from filter import (is_redundant, add,)
from comparator import (std_comparators)

def extend(w:list[Filter], n:int) -> list[Filter]:
    """Extends all filters in a list with a non-redundant comparator in all ways possible.
    N is the channel size of the filters in w"""
    w1 = []
    i = 0
    j = 0
    list_comparator = std_comparators(n)
    comp_length = len(list_comparator)
    filt_length = len(w)
    while i < filt_length:
        while j < comp_length:
            if not is_redundant(list_comparator[j],w[i]):
                f1 = add(list_comparator[j],w[i])
                w1 = w1 + [f1]
                j = j + 1
            j = 0
            i = i + 1
    return w1
```

5.3 Network_finder.py

```
from comparator import *
from network import *
from filter import *
from generate import *
from prune import *

"""Finds a sortingnetwork for n channels by extending an list of of filters
and pruning non-relevant filters"""

print("What should the size of the network be? ")
n = int(input())
l = [make_empty_filter(n)]
count = 0
sortnet = []
while sortnet == []:
    count = count + 1
    i = 0
    l = extend(l,n)
    filtlen = len(l)
    while i < filtlen:
        if is_sorting(l[i]):
            sortnet = sortnet + [l[i]]
            i = i + len(l)
        i = i + 1
    l = prune(l,n)

print("The size of the network is: " +str(n))
print("Number of extends: " +str(count))
print("Size: " +str(size(sortnet[0].net)))
print("Found a sorting network on " +str(n)+ " channels with size " +str(count))
print("The Python implementation for sorting a list v of size " +str(n)+ " is:")
print(to_program(sortnet[0].net,"v","aux"))
```

6 Corrections and notes from fase I

6.1 Functions

```
def is_sorting(net: Network, size:int) -> bool:
    """Checks if net is a sortingnetwork for n input.
    >>>is_sorting([comparator(0,1), comparator(i=0, j=2),
    comparator(i=1, j=2)],[[0, 0, 0], [1, 0, 1], [1, 1, 1]])
    False
    """
    all_combinations = all_outputs(net, size)
    for i in all_combinations
        for j in all_combinations[i]
            if all_combinations[i][j+1] < all_combinations[i][j]:
                return False
    return true

def std_subsets_of_two(v:list) -> list:
    """Returns a index-list of all subsets of a list, with the length of 2,
    that fits as inputs for a standard comparator.
    >>>std_subsets_of_two([1,2,3])
    [[0,2], [0,1], [1,2]]
    """
    result = []
    for i in v:
        j = len(v)-1
        while j > i:
            result.append([i,j])
            j = j - 1
    return result

def std_comparators(n:int) -> list[comparator]:
    """Returns all standard comparators on n channels. n needs to be >=1.
    >>>std_comparators(3)
    [comparator(0,2),comparator(0,1),comparator(1,2)]
    """
    v = std_subsets_of_two(range(n))
    return list(map(lambda x: make_comparator(v[x][0], v[x][1]), range(0,len(v))))

def outputs(net:Network, w:list[list[int]]) -> list[list[int]]:
    """Applies net on all inputs in w and returns a list with no duplicates.
    >>>outputs([comparator(0,1), comparator(1,2), comparator(0,2)],
    [[2,1,3], [1,3,2], [1,3]])
    [1,2,3], [1,3]
    """
    i = 0
    while i < len(w):
        w[i] = apply(net,w[i])
        i = i + 1
    duplikat = []
```

```

w = list(filter(lambda x: (x not in duplikat)
                  and not duplikat.append(x), w))

return w

def append(c:comparator, net:Network) -> Network:
    """Adds a comparator at the end of a network.
    >>>append(comparator(3,4), v[])
    v[comparator(3,4)]
    """
    net.v = net.v + [c]
    return net

def is_standard(net:Network) -> bool:
    """Checks if net only contains standard comparators.
    >>>is_all_standard([comparator(0,1), comparator(1,2), comparator(0,2)])
    True
    """
    i = 0
    result = True
    while i < size(net):
        if is_standard_comparator(net.v[i]) == False:
            result = False
        i = i+1
    return result

```

6.2 Corrected functions

```

def std_subsets_of_two(v:list) -> list:
    """Returns an index-list of all subsets of a list, with the length of 2,
    that fits as inputs for a standard comparator.
    >>>std_subsets_of_two([1,2,3])
    [[0,2], [0,1], [1,2]]
    """
    result = []
    for i in v:
        j = len(v)-1
        while i > j:
            result.append([i,j])
            j = j - 1
    return result

def outputs(net:Network, w:list[list[int]]) -> list[list[int]]:
    """Applies net on all inputs in w and returns a list with no duplicates.
    >>>outputs([comparator(0,1), comparator(1,2), comparator(0,2)],
    [[2,1,3], [1,3,2], [1,3]])
    [1,2,3], [1,3]
    """
    unique_filters = []
    for i in range(len(net))
        w[i] = apply(net,w[i])
        if w[i] not in duplikat:
            unique_filters = unique_filters + w[i]
    return unique_filters

```

```

def append(c:comparator, net:Network) -> Network:
    """Adds a comparator at the end of a network.
    >>>append(comparator(3,4), v[])
    v[comparator(3,4)]
    """

    new_net = net + [c]
    return new_net

def is_standard(net:Network) -> bool:
    """Checks if net only contains standard comparators.
    >>>is_standard([comparator(0,1), comparator(1,2), comparator(0,2)])
    True
    """

    for i in net.v:
        if not is_standard_comparator(net.v[i]):
            result = False
    return True

def is_standard(net:Network) -> bool:
    """Checks if net only contains standard comparators.
    >>>is_all_standard([comparator(0,1), comparator(1,2), comparator(0,2)])
    True
    """

    i = 0
    while i < size(net):
        if is_standard_comparator(net.v[i]):
            return false
        i = i+1
    return true

```

7 Corrections and notes from fase II

```
def is_sorting(f:Filter) -> bool:
    """Checks if the network in the filter is a sortingnetwork. """
    return is_sorting_network(f.net,len(f.output[0]))

def is_sorting_network(net: Network, size:int) -> bool:
    """Checks if net is a sortingnetwork for n input.
    >>>is_sorting([comparator(0,1), comparator(i=0, j=2),
    comparator(i=1, j=2),],[[0, 0, 0], [1, 0, 1], [1, 1, 1]])
    False
    """
    is_sorted = True
    i = 0
    all_combinations = all_outputs(net, size)
    while i < len(all_combinations):
        j = 0
        while j < len(all_combinations[i])-1:
            if all_combinations[i][j+1] < all_combinations[i][j]:
                is_sorted = False
                j = j + 1
            if is_sorted == True:
                i = i + 1
            else:
                return is_sorted
    return is_sorted

def is_sorting(f:Filter) -> bool:
    """Checks if the network in the filter is a sortingnetwork. """
    for i in f.output:
        for j in f.output[i]:
            if f.output[i][j+1] < f.output[i][j]:
                return False
    return True
```

```

"""Finds a sortingnetwork for n channels by extending an list of of filters
and pruning non-relevant filters"""

print("What should the size of the network be? ")
n = -1
while type(n) is not int or n <= 1:
    print("Please type a positive integer above 1: ")
    try:
        n = int(input())
    except ValueError:
        n = -1
l = [make_empty_filter(n)]
count = 0
sortnet = []
while sortnet == []:
    count = count + 1
    i = 0
    l = extend(l,n)
    if count > 0:
        l = prune(l,n)
    while i < len(l):
        if is_sorting(l[i]):
            sortnet = sortnet + [l[i]]
            i = i + len(l)
        i = i + 1

print("The size of the network is: " +str(n))
print("Number of extends: " +str(count))
print("Size: " +str(size(sortnet[0].net)))
print("Found a sorting network on " +str(n)+ " channels with size " +str(count))
print("The Python implementation for sorting a list v of size " +str(n)+ " is:")
print(to_program(sortnet[0].net,"v","aux"))

@dataclass
class Network:
    v: list[comparator]

Network = list[comparator]

@dataclass
class Comparator:
    i: int
    max: int

@dataclass
class Comparator:
    min: int
    max: int

```


8 Funktioner

"""Finds a sortingnetwork for n channels by extending an list of of filters and pruning non-relevant filters"""

```
print("What should the size of the network be? ")
n = -1
while type(n) is not int or n < 1:
    print("Please type a positive integer above 1: ")
    try:
        n = int(input())
    except ValueError:
        n = -1
l = [make_empty_filter(n)]
f1 = calculate_score(l,n)
l = extend(l,n)
l = second_prune(l,n)
f2 = calculate_score(l,n)

count = 0
sortnet = []
t1 = time.time()
while sortnet == []:
    count = count + 1
    i = 1
    pv = predicted_value(f1,f2,n)
    l = extend_until_score(l,n,pv,3)

    while i < len(l):
        if is_sorting(l[i]):
            sortnet = sortnet + [l[i]]
            i = i + len(l)
        i = i + 1

    f1 = f2
    f2 = calculate_score(l,n)

t2 = time.time()
```

```

def second_prune2(w:list[Filter],n: int)-> list[Filter]:
    """Scores filters in list w and returns a new list with on n channels from w"""
    i = 0
    score = []
    while i < len(w):
        del_list = []
        sorted_list = identical_sorted_list(w[i].output,n)
        j = 0
        count = 0
        count_last = 0
        while j < len(w[i].output):
            k = 0
            while k < n:
                if w[i].output[j][k] != sorted_list[j][k]:
                    count = count + 1
                    k = k + 1
            if count == count_last:
                del_list = del_list + [j]
                count_last = count
            j = j + 1
        score = score + [int(count/2)]
        w[i].output = del_from_list(del_list,w[i].output)
        w[i].output = output_duplicates(w[i].output)
        i = i + 1
    return keep_percentage_filters(w,score,5)

def third_prune(w:list[Filter], n: int) -> list:
    """Prunes filters in a list (w) of filters on n channels by giving all filters
    a score based on perfomance, and removing all filters
    under with a score less than the average."""
    score = []
    index = range(len(w[0].output[0]))
    i = 0
    while i < len(w):
        j = 0
        count = 0
        while j < len(w[0].output):
            count = count - reduce(lambda x, y: x + (y**2), ([k for k in index
                if w[i].output[j][k] == 1] + [0]))
            j = j + 1
        score = score + [count]
        i = i + 1
    return keep_percentage_filters(w,score,17)

```

```

def second_prune(w:list[Filter],n: int)-> list[Filter]:
    """Scores filters in list w and returns a new list with on n channels from w"""
    i = 0
    score = []
    while i < len(w):
        sorted_list = identical_sorted_list(w[i].output,n)
        j = 0
        count = 0
        while j < len(w[0].output):
            k = 0
            while k < n:
                if w[i].output[j][k] != sorted_list[j][k]:
                    count = count + 1
                    k = k + 1
                j = j + 1
            score = score + [int(count/2)]
            i = i + 1
    return keep_percentage_filters(w,score,1)

def extend(w:list[Filter], n:int) -> list[Filter]:
    """Extends all filters in a list with a non-redundant comparator in all ways possible"""
    w1 = []
    i = 0
    list_comparator = std_comparators(n)
    comp_length = len(list_comparator)
    filt_length = len(w)
    while i < filt_length:
        j = 0
        while j < comp_length:
            if not is_redundant(list_comparator[j],w[i]):
                f1 = make_empty_filter(n)
                f1 = add(list_comparator[j],w[i])
                w1 = w1 + [f1]
            j = j + 1
        i = i + 1

    return w1

```